



2.8. Stream API и Optional

Привет!

На связи шпаргалка урока 2.8. Stream API и Optional. В этом материале мы разберем появившийся в 8-й версии Java альтернативный способ по работе с данными — Stream API. Также изучим тип Optional, предназначенный для борьбы с NullPointerException.

Время прочтения: 30 минут.

Для усвоения этих тем потребуется еще раз взглянуть на лямбда-выражения. Мы уже вскользь упоминали лямбда-выражения в шпаргалке урока 2.7. Коллекции, generics, map. Сегодня разберем их подробнее.

[Лямбда-выражения](#)

[Optional](#)

[Создание Optional](#)

[Методы Optional](#)

[Проверка значения](#)

[Манипуляции со значением](#)

[Stream API](#)

[Создание стрима](#)

[Создание стрима на основе имеющихся данных](#)

[Генерация стрима](#)

[Создание стрима на основе интервала значений](#)

[Операторы: терминальные и конвейерные](#)

[Конвейерные \(промежуточные\) операторы](#)

[Терминальные операторы](#)

Лямбда-выражения

Лямбда-выражения представляют собой анонимную реализацию какого-то метода. Разберем синтаксис таких выражений.

1. Без аргумента.

```
() -> System.out.println("Hello")
```

2. Один аргумент.

```
s -> System.out.println(s)
```

3. Два аргумента.

```
(x, y) -> x + y
```

4. С открытым типом аргументов.

```
(Integer x, Integer y) -> x + y
```

5. Множественный оператор.

```
(x, y) -> {  
    System.out.println(x);  
    System.out.println(y);  
    return (x + y);  
}
```

Лямбда-выражения имеют два блока, которые разделены стрелкой (**->**). До стрелки указываются параметры метода, а после — тело метода. Если строка в теле метода одна, мы можем не оборачивать его в скобки. Если строк больше, по принятому стандарту описания методов нужно обернуть код в **{ }** и сделать **return** в конце блока, но это требуется не всегда. Зависит от того, какую конкретно лямбду хочет получить метод в качестве параметра.

Рассмотрим на примерах.

Лямбда-выражение вида **() -> System.out.println("Hello")** можно представить в виде следующего метода:

```
void doSmth() {  
    System.out.println("Hello");  
}
```

Лямбда-выражение вида **(Integer x, Integer y) -> x + y** можно представить в виде такого метода:

```
void doSmth(Integer x, Integer y) {  
    return x + y;  
}
```

Если же строк в лямбда-выражении больше одной, то метод изменится таким образом:

```
(Integer x, Integer y) -> {  
    System.out.println(x);  
    System.out.println(y);  
    return x + y;  
}
```

Выглядит в привычном восприятии так:

```
void doSmth(Integer x, Integer y) {  
    System.out.println(x);  
    System.out.println(y);  
    return x + y;  
}
```

Заметьте, что на фрагментах выше тип параметров указывается не всегда. На самом деле, любая из переменных, что объявляется в блоке до стрелки, имеет тип. Это происходит потому, что лямбда-выражения в некоторых случаях позволяют нам «выявить» тип на основе контекста, где применяется лямбда-выражение.

Вспомним шпаргалку урока 2.6, где мы работали с коллекциями с помощью методов **removeIf** и **replaceAll**, которые в качестве параметра принимали лямбда-выражение.

```
nums.removeIf(e -> e % 2 == 1);
```

В данном случае из контекста понятно, что **e** является элементом коллекции **nums**, т. е. числом. Поэтому явное указание типа можно опустить.

В ином случае нам бы пришлось писать **(Integer e) -> e % 2 == 1**

```
nums.removeIf((Integer e) -> e % 2 == 1);
```

В определенных случаях от лямбда-выражения требуется вернуть результат. Так в примере выше с **removeIf** нам нужно вернуть из лямбда-выражения boolean-результат. На основе этого результата будет решено, удалять элемент или нет.

В некоторых случаях IDEA предложит вам изменить конструкцию выше на конструкцию вида **тип::имя_метода**. Эта конструкция говорит, что нужно вызвать определенный метод определенного класса, передав в него те параметры, что имеет лямбда-выражение.

Подобные предложения от IDEA всплывают в ситуациях, когда в стандартной библиотеке Java уже есть метод, выполняющий аналогичный функционал.

Например:

(String v) -> System.out.println(v) будет предложено заменить на **System.out::println**

(Integer x, Integer y) -> x + y будет предложено заменить на **Integer::sum**

Optional

Так как в Java количество NullPointerException превышает общее количество всех вместе взятых исключений, выбрасываемых приложениями нужно было что-то решать. Именно для этого был придуман тип Optional.

Тип Optional представляет собой некий контейнер, где может быть элемент, а может быть null.

Мы возвращаем из метода этот контейнер, а уже принимающая сторона обрабатывает его так, как ей нужно.

В зависимости от содержимого контейнера, проверка которого на **null** заложена в типе Optional «из коробки», будет вызван определенный метод, возвращающий дефолтный результат или выбрасывающий нужное исключение.

Это станет понятнее при разборе методов Optional далее.

Рассмотрим, как создать Optional.

Создание Optional

Создать Optional можно несколькими способами.

1. Статический метод of(T value).

Этот способ позволяет создать объект Optional на основе какого-то объекта, который не равен null.

Пример:

```
String result = "result of method";  
return Optional.of(result);
```

2. Статический метод ofNullable(T value).

Способ позволяет нам создать объект Optional на основе какого-то объекта, который может быть как равен, так и не равен null.

Мы можем избежать проверки.

Пример:

```
String result = null;  
return Optional.ofNullable(result)
```

3. Статический метод empty().

Этот способ гарантированно возвращает `Optional`, который «заполнен» значением `null`.

Пример:

```
return Optional.empty();
```

Методы `Optional`

Проверка значения

`isPresent()/isEmpty()` — методы проверяют, что значение в `Optional` равно / не равно `null`, и возвращают `boolean`-результат.

`ifPresent()` — метод принимает лямбда-выражение и выполняет его в том случае, если значение не равно `null`.

Пример:

```
Optional<String> optional = Optional.of("result of method");
optional.ifPresent(value -> System.out.println(value));
```

Или можно заменить лямбду выше ссылкой на метод.

```
optional.ifPresent(System.out::println);
```

Работа с `null`

`orElse()` — метод принимает еще одно значение, дефолтное, того же типа, что указан в `generic` полученного `Optional`, и в случае значения `null` внутри `Optional` возвращает это дефолтное значение.

Пример:

```
Optional<String> optional = Optional.empty();
System.out.println(optional.orElse("default string")); // напечатает default string

Optional<String> optional2 = Optional.of("result of method");
System.out.println(optional2.orElse("default string")); // напечатает result of method
```

`orElseThrow()` — метод выбрасывает исключение в тех случаях, когда значение `null`.

Если скобки при вызове остаются пустыми, выбрасывается `NoSuchElementException`.

Когда нужно выбросить какое-то свое исключение, можно написать следующее:

```
optional.orElseThrow(() -> new RuntimeException("Value is null"));
```

Если дополнительное сообщение указывать не требуется, можно использовать ссылку на `new`.

```
optional.orElseThrow(RuntimeException::new)
```

`orElseGet()` — метод вызывает блок кода в случае, когда значение равно `null`.

Например, мы хотим вызвать другой метод, который сформирует результат этого же типа.

Код будет выглядеть так:

```
Optional<String> optional = Optional.empty();
optional.orElseGet(() -> generateRandomString());
```

```
// или, если класс со статическим методом generateRandomString называется Main,  
// можно написать так:  
  
optional.orElseGet(Main::generateRandomString)
```

В тот момент, когда в нашем `Optional` будет `null`, произойдет вызов метода `generateRandomString`, и уже его результат будет возвращен в качестве значения `Optional`.

Манипуляции со значением

`get()` — возвращает значение. Если значения нет, бросает `NoSuchElementException`.

Логика такая же, как у `orElseThrow` без дополнительных параметров в скобках.

`map()` — возвращает новый `Optional`, в котором преобразовывает значение в другой объект на основе лямбда-выражения.

Например:

```
Optional<Integer> optional = Optional.of(3);  
  
Optional<Integer> mappedOptional = optional.map(v -> v + 1);  
  
Optional<String> mappedOptional2 = optional.map(v -> v.toString());  
  
Optional<String> mappedOptional3 = optional.map(Object::toString);
```

В коде выше показаны три разных способа применения.

Первый увеличит значение из `Optional` на 1.

Второй и третий являются разными записями одной реализации.

Их задача — конвертировать число 3 в строку.

В результате в `mappedOptional2` и 3 будет лежать строка “3”.

`filter()` — проверяет, что значение соответствует условию из лямбда-выражения. Если проверка проходит, то метод возвращает `Optional` с этим значением. Иначе возвращает пустой (`Optional.empty()`).

Например:

```
Optional<Integer> optional = Optional.of(3);  
  
Optional<Integer> filteredOptional = optional.filter(e -> e % 3 == 0); // Optional.of(3);  
  
Optional<Integer> filteredOptional2 = optional.filter(e -> e % 4 == 0); // Optional.empty()
```

`flatMap()` — очень похож на метод `map`, но работает только в том случае, когда результат лямбда-выражения не равен `null`.

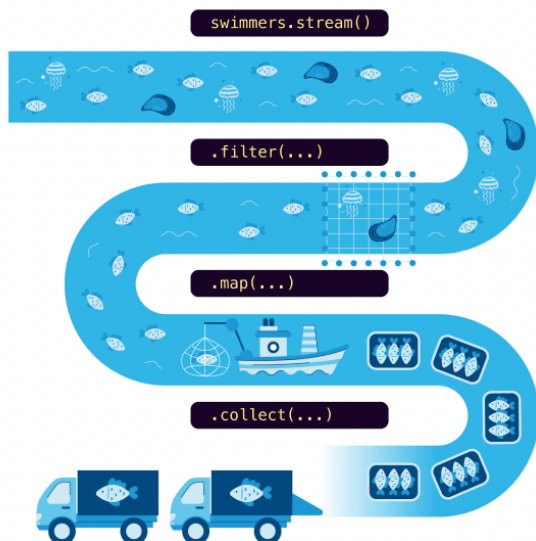
Stream API

Stream API является еще одним способом по работе с данными. Самый популярный источник данных — коллекции.

Однако данные можно и генерировать «на лету» с помощью специальных методов Stream. Во время выполнения Stream проводит определенную цепочку действий с каждым значением, а затем в финальном методе цепочки собирает эти данные (после всех модификаций) в коллекцию, строку или просто печатает в консоль.

Рассмотрим пример работы Stream API.

Фильтрация списка



Фильтрация списка помогает писать существенно меньше кода.

На картинке изображен следующий код:

```
List<Swimmer> swimmers = List.of("набор плавающих существ");

SeaProductsFactory factory = new SeaProductsFactory();

List<FishPack> fishPacks = swimmers.stream() // Создаем stream на основе коллекции
    .filter(swimmer -> swimmer.isFish()) // Отсеиваем элементы, которые не рыбы
    .map(fish -> factory.pack(fish)) // Упаковываем рыбу в контейнеры
    .collect(Collectors.toList()); // Собираем контейнеры в новый список
```

Код получился коротким и функциональным. За пару строк мы отфильтровали и конвертировали все данные из списка. В случае работы через циклы нам бы пришлось работать с кодом, который значительно больше представленного выше.

Но Stream API не панацея. Практически всегда работа через Stream API выполняется ради читаемости и поддерживаемости кода. Если оценивать производительность и затраты ресурсов, циклы абсолютно точно будут быстрее и легче.

Создание стрима

Стрим можно создать на основе имеющихся данных (коллекция, массив или просто набор элементов) или сгенерировать на основании определенного действия.

Создание стрима на основе имеющихся данных

Чтобы создать Stream на основе данных из коллекции, нужно вызвать у коллекции метод `stream()`. Этот метод реализован у всех списков и множеств: `list.stream()` или `set.stream()`

В случае работы с картой требуется получить множество пар «ключ-значение» и уже у него вызвать метод `stream()`: `map.entrySet().stream()`.

Для создания стрима на основе массива нужно вызвать соответствующий метод класса `Arrays`: `Arrays.stream(arr)`.

Чтобы создать стрим на основе произвольных элементов, нужно использовать метод `of`:

```
Stream.of(1, 2, 3)
```

Генерация стрима

Стрим можно сгенерировать несколькими способами. Рассмотрим 2 самых популярных варианта. Это методы интерфейса `Stream` `generate` и `iterate`.

generate() — метод принимает лямбда-выражение, которое должно вызвать определенный код, возвращающий значение определенного типа. Именно этими значениями и будет заполнен стрим.

Например:

```
Stream.generate(() -> 10); // В этом случае будет создан объект Stream, заполненный бесконечным количеством значений 10

Stream.generate(() -> generateRandomNumber()); // В этом случае стрим будет заполнен случайными числами, которые сгенерирует наш метод gene
```

iterate() — метод принимает два параметра. Первый является «стартовым» значением, второй — лямбда-выражением, которое содержит модификацию предыдущего элемента стрима.

Например:

```
Stream.iterate(2, e -> e * 2); // В этом случае будет создан стрим, который первым элементом будет содержать 2, а каждый следующий элемент
```

Создание стрима на основе интервала значений

Для примитивных типов существуют отдельные версии `Stream`. Например, `IntStream` и `LongStream`.

Эти интерфейсы позволяют создавать стримы на основе интервала значений.

Для этого используются методы `range` и `rangeClosed`.

range() — метод принимает два параметра, которые являются стартовым и заключительным элементом. Заключительный элемент в стрим не включается.

Например:

```
IntStream.range(1, 5); // Будет создан стрим с элементами 1, 2, 3 и 4
```

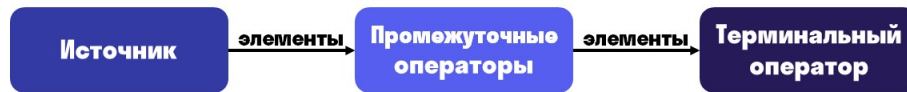
rangeClosed() — метод работает так же, как `range()`, но второй параметр включается в стрим.

Например:

```
IntStream.rangeClosed(1, 5); // Будет создан стрим с элементами 1, 2, 3, 4 и 5
```

Операторы: терминальные и конвейерные

Стримы считаются «ленивыми». Это значит, что никаких манипуляций с данными не происходит до того, как цепочка не будет «замкнута» терминальным оператором.



Терминальными операторами являются методы, завершающие цепочку действий с данными. Например, метод `collect` из примера выше является терминальным, так как он собирает элементы в какой-то итоговый результат, будь то коллекция, массив или строка. Никаких действий после него выполнено быть не может, так как стрим в этот момент конвертируется в коллекцию. После того как был вызван терминальный метод, стрим закрывается. Повторный вызов терминального метода невозможен.

Конвейерными операторами являются методы, производящие действия с данными. Например, `filter` и `map` из примера выше. Эти методы возвращают модифицированный объект `Stream`, позволяя этим вызывать следующий конвейерный или терминальный метод, строя своего рода цепочку действий.

Следуя логике, описанной выше, вы вполне можете создавать разные стримы на основе одних и тех же данных, дополняя или убирая действия из цепочки.

Например:

```
IntStream nums = IntStream.range(1, 5);

if (evenNeeded) {
    nums = nums.filter(e -> e % 2 == 0);
}

if (needToMapNums) {
    nums = nums.map(e -> e * 2);
}

nums.forEach(System.out::println);
```

В примере мы в зависимости от флагов фильтруем и модифицируем элементы стрима. Если оба условия окажутся `true`, значения будут модифицированы двумя конвейерными операциями. Если какое-то из условий окажется `false`, значения будут модифицированы одним или даже нулем конвейерных операций.

`forEach` — еще один пример терминального оператора. Принимает в себя лямбду и выполняет схожую задачу, что и цикл `forEach`.

Конвейерные (промежуточные) операторы

Разберем самые популярные примеры конвейерных операций.

filter() — метод принимает лямбду, которая обязана вернуть boolean-результат.

Примеры с фильтрацией уже были выше.

map() — метод принимает лямбду, которая модифицирует каждое значение в стриме.

Следует учесть, что если имеется какое-то ветвление (оператор if), то лучше предварительно использовать метод filter.

Например:

```
List<String> numbers = Stream.of(1, 2, 3)
    .map(v -> v.toString)
    .collect(Collectors.toList());
```

Этот пример конвертирует набор чисел в строки, а затем собирает их в список строк.

Есть альтернативы метода map, которые конвертирует стрим в упомянутые выше стримы примитивов: mapToInt, mapToLong и mapToDouble.

Примитивные стримы имеют метод mapToObj, конвертирующие примитивы в объекты.

skip() — метод принимает один параметр, указывающий на количество элементов, которые нужно пропустить.

Например:

```
Set<Integer> nums = IntStream.range(1, 10)
    .skip(5)
    .forEach(System.out::println);
```

Здесь первые 5 элементов будут пропущены, а остальные элементы будут напечатаны в консоль.

limit() — метод является полной противоположностью метода skip(). Он берет из стрима то количество элементов, что указано в качестве параметра.

Например, таким способом можно ограничить количество сгенерированных элементов метода generate:

```
List<Integer> nums = Stream.generate(() -> 10)
    .limit(50)
    .collect(Collectors.toList());
```

Если мы теперь распечатаем размер списка nums, то увидим, что в нём всё соответствует заданным лимитам.

sorted() — метод имеет два варианта. Если не передать параметр, сортировка будет осуществлена в натуральном порядке, т. е. от меньшего к большему. Если нужен другой алгоритм сортировки, требуется в качестве параметра передать лямбду или объект типа Comparator.

Например:

```
Stream.of(1, 5, 2, 10)
    .sorted(Comparator.reverseOrder())
    .forEach(System.out::println);
```

В примере будет произведена сортировка в обратную сторону, т. е. от большего к меньшему, а затем печать в консоль.

distinct() — метод исключает из стрима повторяющиеся элементы.

Например:

```
Stream.of(1, 1, 1, 2)
    .distinct()
    .forEach(System.out::println);
```

Будут напечатаны только 1 и 2, а дубли удалены.

Терминальные операторы

Два из терминальных операторов мы уже обсудили выше.

forEach() — метод принимает лямбду, которая не требует возвращения результата. Например, печать, которая была показана в примерах выше.

collect() — метод принимает лямбду или объект типа Collector.

Примеры были выше, но самыми популярными вариантами являются следующие:

```
.collect(Collectors.toList()) -- собирает стрим в список
.collect(Collectors.toSet()) -- собирает стрим в сет
.collect(Collectors.toMap(e -> e, e -> e.toString())) -- собирает стрим в mapу
```

Метод toMap() принимает два параметра. Первый параметр отвечает за генерацию ключа, второй — за генерацию значения. В примере выше мы ключом положили элемент нашего стрима, а значением — его строчное представление.

.collect(Collectors.joining()) — собирает строки из стрима в общую строку. Если есть другие элементы, предварительно нужно вызвать **map (e -> e.toString())**

count() — метод возвращает количество элементов в стриме.

min()/max() — методы возвращают минимальное или максимальное значение, обернутое в Optional. В качестве параметра принимают объект типа Comparator или лямбду, аналогичную методу sorted.

Например:

```
int max = Stream.of(1, 5, 2, 10)
    .max(Comparator.naturalOrder())
    .get();
```

Переменной max будет присвоено значение 10.

```
int min = Stream.of(1, 5, 2, 10)
    .min(Comparator.naturalOrder())
    .get();
```

Переменной min будет присвоено значение 1.

sum() — метод примитивных стримов. Возвращает сумму элементов.

Например:

```
int sum = IntStream.of(1, 2, 3)
    .sum();
```

Переменная sum получит значение 6.

average() — метод примитивных стримов. Возвращает среднее значение в OptionalDouble.

Пример:

```
double avg = IntStream.of(1, 5, 2, 10)
    .average()
    .getAsDouble();
```

Переменная avg получит в себя значение 4.5.

findFirst()/findAny() — возвращают первый элемент стрима. В случае, когда стримы выполняются многопоточно, `findAny` и `findFirst` могут вернуть разный результат.

`findFirst` гарантированно вернет первый элемент стрима, когда `findAny` может вернуть первый элемент конкретного потока, что не всегда является первым элементом стрима.

allMatch()/anyMatch()/noneMatch() — методы в качестве параметра получают логическое выражение, возвращающее `true/false`, затем пропускают все элементы через это логическое выражение.

Разница в том, что `allMatch` вернет `true` только в том случае, когда все элементы прошли через выражение и вернули `true`. Например, все числа четные.

`anyMatch` вернет `true`, если хотя бы один элемент прошел через логическое выражение и вернул `true`. Например, как минимум одно число является четным.

`noneMatch` вернет `true` в случае, если все элементы прошли через логическое выражение и вернули `false`. Например, все числа не являются четными.

```
// Будет false, так как число 1 не является четным. Дальше проверка не имеет смысла
boolean allEven = IntStream.of(1, 5, 2, 10)
    .allMatch(e -> e % 2 == 0);

// Будет true, так как есть число 2, которое является четным. Дальше проверка не имеет смысла
boolean anyEven = IntStream.of(1, 5, 2, 10)
    .anyMatch(e -> e % 2 == 0);

// Будет false, так как число 2 является четным. Дальше проверка не имеет смысла
boolean noneEven = IntStream.of(1, 5, 2, 10)
    .noneMatch(e -> e % 2 == 0);
```

Шпаргалка урока в PDF-формате:

https://s3-us-west-2.amazonaws.com/secure.notion-static.com/d7dcba6c-6a0a-49cc-80bd-0271a7e689a8/Шпаргалка_2.8._Stream_Api_и_optional.pdf